Codes Correcteurs d'erreurs

Implémentation de Reed-Solomon

Nasr-allah HITAR Khalil BITAR

ENSEIRB-MATMECA

30 Mars 2026

Encadrant : Camille LEROUX

30 MARS 2026

Sommaire

1. Introduction – Codes correcteurs
2. Codes de Reed-Solomon
3. Implémentation
4. Tests et Résultats
5. Vectorisation des instructions SIMD
6. Annexe

Notion de code correcteur

Définitions

- L'**alphabet** : ensemble fini de symboles avec lesquels est écrit le message. On prendra $\mathrm{GF}(2^m)$ (corps fini à 2^m éléments).
- Le **message** : élément de $\mathrm{GF}(2^m)^k$ (vecteur de k symboles) ou polynôme de degré $< k$.
- L'**encodeur** : fonction injective enc : $\mathrm{GF}(2^m)^k \to \mathrm{GF}(2^m)^n$.
- Le **code** : sous-ensemble de $\mathrm{GF}(2^m)^n$: $\mathcal{C} = \mathrm{Im}(\mathrm{enc})$.
- Le **décodeur** : fonction dec : $\mathrm{GF}(2^m)^n \to \mathrm{GF}(2^m)^k$.

Étude algorithmique

On cherche un code $\mathcal{C}$ optimal pour des valeurs n et k fixés.
Expliciter enc et dec pour pouvoir les calculer rapidement.

Au fait qu'est-ce qu'un corps de Galois (GF) ?

Définition "vulgarisée"

Un **corps de Galois** GF(q) est un ensemble fini d'éléments sur lequel on peut **additionner, soustraire, multiplier et diviser** (sauf par zéro), avec les mêmes règles que l'arithmétique usuelle (associativité, commutativité, distributivité). La particularité : il ne contient qu'un nombre q d'éléments, et q est toujours une puissance d'un nombre premier : $q = p^m$.

Exemple concret : GF(2)

GF(2) $= \{0, 1\}$. Les opérations sont :

- Addition : $0 + 0 = 0$, $0 + 1 = 1$, $1 + 0 = 1$, $1 + 1 = 0$ (XOR, ou "ou exclusif").
- Multiplication : $0 \times 0 = 0$, $0 \times 1 = 0$, $1 \times 0 = 0$, $1 \times 1 = 1$ (ET logique).

C'est le corps des bits, utilisé dans la plupart des codes correcteurs.

Pourquoi cette structure ?

Pourquoi GF(2^m) dans Reed–Solomon ?

On utilise GF(2^m) car :

- Chaque élément peut être représenté par un octet ($m = 8$) ou un mot binaire.
- On dispose de $2^m - 1$ éléments non nuls, ce qui permet de construire des codes de longueur $n = 2^m - 1$.
- L'addition est un simple XOR (rapide), la multiplication se fait via des tables précalculées.

Codes t-correcteurs

Distance et poids

Pour $(c_1, c_2) \in (\mathrm{GF}(2^m)^n)^2$ on définit la **distance de Hamming** par

$$d(c_1, c_2) = \sum_{i=1}^{n} \big(1 - \delta(c_{1,i}, c_{2,i})\big).$$

Le **poids** est la distance au mot nul : $w(c_1) = d(c_1, 0)$.

- C'est le nombre de symboles distincts entre deux mots du code.
- Dans $(\mathbb{F}_2)^4$, on a $d(1001, 1011) = 1$.

Codes t-correcteurs (suite)

Pour $\mathcal{C}$ un code, $c \in \mathcal{C}$ et t un entier, on pose

$$E(c,t) = \left\{ m \in \mathrm{GF}(2^m)^n \mid d(c,m) \leq t \right\}.$$

Il s'agit d'une boule centrée en c de rayon t.

Définition

Pour $t \in \mathbb{N}$, on dit qu'un code $\mathcal{C}$ est t-**correcteur** lorsque les ensembles $(E(c,t))_{c \in \mathcal{C}}$ sont deux-à-deux disjoints.
Si moins de t erreurs se sont produites, il existe alors un unique mot du code le plus proche.

Distance minimale et correction

Distance minimale

Pour $\mathcal{C}$ un code, on pose

$$d_{\min} = \min_{\substack{(c_1,c_2)\in\mathcal{C}^2 \\ c_1\neq c_2}} d(c_1, c_2).$$

Theorem

Un code $\mathcal{C}$ de distance minimale $d_{\min}$ est $\left\lfloor \frac{d_{\min}-1}{2} \right\rfloor$-correcteur mais n'est pas $\left\lfloor \frac{d_{\min}-1}{2} \right\rfloor + 1$-correcteur.

Codes linéaires

Un code $\mathcal{C}$ est **linéaire** si $\mathcal{C}$ est un sous-espace vectoriel de $\mathrm{GF}(2^m)^n$.

Distance d'un code linéaire

Pour $\mathcal{C}$ linéaire, on a

$$d_{\min} = \min_{c \in \mathcal{C} \setminus \{0\}} w(c).$$

Majoration de Singleton

Tout code linéaire de paramètre (n, k) vérifie

$$d_{\min} \leq n - k + 1.$$

Paramètres de Reed-Solomon

- Alphabet : $\mathrm{GF}(2^m)$.
- Paramètres :
  - n la longueur du code,
  - k sa dimension (longueur du message initial),
  - $2t$ le nombre de symboles ajoutés.
- Les paramètres doivent vérifier $n = k + 2t = 2^m - 1$.

Message : on identifie $\mathrm{GF}(2^m)^k$ à $\mathrm{GF}(2^m)_{<k}[X]$ (polynômes de degré $< k$).
La liste $(a_0, \ldots, a_{k-1})$ représente le polynôme $a_0 + a_1 X + \cdots + a_{k-1} X^{k-1}$.

Code de Reed-Solomon

Soient $\alpha_1, \ldots, \alpha_{2t}$ des éléments distincts de $\mathrm{GF}(2^m)$. On pose

$$G(X) = \prod_{i=1}^{2t} (X - \alpha_i).$$

Le **code de Reed-Solomon** est l'ensemble des multiples de G :

$$\mathcal{C} = \left\{ G(X)P(X) \mid P \in \mathrm{GF}(2^m)_{<k}[X] \right\}.$$

Principe de la démonstration (distance minimale)

Soit $c = \sum_i c_i X^i \in \mathcal{C}$ tel que $w(c) \leq 2t$. Les α_i $(i = 1, \ldots, 2t)$ étant racines de c, on a le système

$$\begin{bmatrix} \alpha_1^1 & \cdots & \alpha_{2t}^1 \\ \vdots & \ddots & \vdots \\ \alpha_1^{2t} & \cdots & \alpha_{2t}^{2t} \end{bmatrix} \begin{bmatrix} c_{i_1} \\ \vdots \\ c_{i_{2t}} \end{bmatrix} = \begin{bmatrix} 0 \\ \vdots \\ 0 \end{bmatrix}.$$

La matrice est de Vandermonde, donc inversible $\Rightarrow c = 0$. Ainsi $d_{\min} \geq 2t + 1 = n - k + 1$ (Singleton atteint).

Encodeur

On veut transformer $A \in \mathrm{GF}(2^m)_{<k}[X]$ en un multiple de G de degré $< n$.

Algorithme d'encodage systématique

1. $G \leftarrow \prod_{i=1}^{2t}(X - \alpha_i)$
2. $T \leftarrow A \cdot X^{2t}$
3. $B \leftarrow$ reste de la division de T par G (degré $< 2t$)
4. renvoie $T - B$

Ainsi $(a_0, \ldots, a_{k-1})$ devient après codage ce qu'on l'appele la parité du code

$$(\underbrace{m_1, \ldots, m_{2t}}_{\text{symboles ajoutés}},\ a_0, \ldots, a_{k-1}).$$

Correction d'erreur

On suppose que moins de t erreurs se sont produites. Le message reçu s'écrit $D = C + E$ avec E le polynôme d'erreur :

$$E = \sum_{\ell=1}^{\nu} e_{i_\ell} X^{i_\ell}, \qquad \nu \leq t.$$

- Inconnues : nombre d'erreurs ν, positions $(i_1, \ldots, i_\nu)$, valeurs $(e_{i_1}, \ldots, e_{i_\nu})$.
- Pour $\ell = 1, \ldots, 2t$ on a $D(\alpha_\ell) = C(\alpha_\ell) + E(\alpha_\ell) = E(\alpha_\ell)$.

On connaît donc $2t$ valeurs prises par E.

Équation-clé

On définit

$$S(X) = \sum_{\ell=1}^{2t} D(\alpha_\ell) X^{\ell-1}, \qquad \Lambda(X) = \prod_{\ell=1}^{\nu} (1 - \alpha^{i_\ell} X), \qquad \Omega(X) = \sum_{\ell=1}^{\nu} e_{i_\ell} \alpha^{i_\ell} \prod_{h \neq \ell} (1 - \alpha^{i_h} X).$$

Équation-clé

Ces polynômes vérifient

$$\Lambda(X) S(X) \equiv \Omega(X) \pmod{X^{2t}}.$$

(On peut la résoudre par l'algorithme d'Euclide étendu, en éliminant les monômes de degré supérieur à $2t$.)

Détermination des valeurs et positions d'erreurs

Une fois Λ et Ω connus :

- $\nu = \deg(\Lambda)$,
- les racines de Λ sont les α^{-i_ℓ} (donc les positions i_ℓ),
- les valeurs d'erreur sont données par la **formule de Forney** :

$$e_{i_\ell} = -\frac{\Omega(\alpha^{-i_\ell})}{\Lambda'(\alpha^{-i_\ell})}.$$

Représentation du corps de Galois

Classe `GaloisField`

- Polynôme primitif $P(X)$ prédéfini selon m (table `poly_masks`).
- Tables précalculées :
  - `alpha_to[i]` $= \alpha^i$ (forme polynomiale)
  - `index_of[val]` $= i$ tel que $\alpha^i = val$
- Multiplication : $\alpha^a \cdot \alpha^b = \alpha^{(a+b) \bmod (2^m-1)}$

```
inline int mul(int a, int b) {
if (!a || !b) return 0;
return alpha_to[(index_of[a] + index_of[b]) % (size-1)];
}
```

- Addition = XOR ($\mathbb{F}_{2^m}$)
- Toutes les opérations sont ainsi en $O(1)$.

Construction du polynôme générateur

Générateur de Reed–Solomon

$$G(X) = \prod_{i=1}^{2t} (X - \alpha^i)$$

- $2t = n - k$ (nombre de symboles de parité)
- α est un élément primitif de GF(2^m)
- Implémentation : multiplication successive par $(X - \alpha^i)$

```
generator = {1};
for (int i = 1; i <= n-k; i++) {
int alpha_i = gf.get_alpha_to()[i];
generator = poly_mult_by_binomial(generator, alpha_i);
}
```

Précalcul de la table de multiplication pour l'encodeur

Motivation

Dans la boucle d'encodage, on calcule gf.mul(x, generator[j]) très souvent. On précalcule toutes les valeurs possibles dans une table pour accélérer.

```
int r = n - k;
mul_table.resize(r * (n + 1));
for (int j = 0; j < r; j++) {
for (int x = 0; x < n + 1; x++) {
mul_table[j*(n+1) + x] = gf.mul(x, generator[j]);
}
}
```

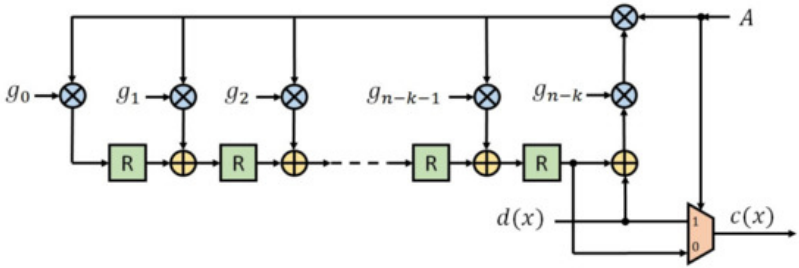
- Tableau 2D aplati : accès en $O(1)$ sans appel de fonction
- x parcourt toutes les valeurs possibles (0 à $2^m - 1$)
- Utilisée dans l'encodeur LFSR

Encodeur systématique par LFSR

Principe

On veut obtenir $[P_0, \ldots, P_{r-1}, m_0, \ldots, m_{k-1}]$ avec P le reste de $X^r M(X)$ par $G(X)$.

- LFSR à r étages, initialisé à 0
- On injecte les symboles du message (de m_{k-1} à m_0)
- À la fin, le registre contient le reste (les symboles de parité)



Algorithme d'encodage

Algorithm 1 Encodeur systématique Reed–Solomon

Require: Message $M = (m_0, \ldots, m_{k-1})$

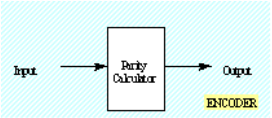
Ensure: Mot de code $C = (p_0, \ldots, p_{r-1}, m_0, \ldots, m_{k-1})$

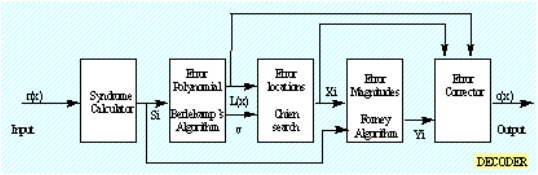
```
r ← n − k
parity ← tableau de longueur r initialisé à 0
for i = k − 1 downto 0 do
    feedback ← m_i ⊕ parity[r − 1]
    for j = r − 1 downto 1 do
        parity[j] ← parity[j − 1] ⊕ mul_table[j · (n + 1) + feedback]
    end for
    parity[0] ← mul_table[feedback]
end for
C ← parity concaténé avec M
return C
```

- $\oplus$ désigne l'addition dans GF(2^m) (XOR)
- `mul_table` permet un accès direct au produit feedback $\times\, g_j$

Décodeur : aperçu (flux de traitement)

1. **Calcul des syndromes** $S_i = D(\alpha^i)$ pour $i = 1, \ldots, 2t$
2. **Algorithme de Berlekamp–Massey** : trouve le polynôme localisateur d'erreurs $\Lambda(X)$
3. **Recherche de Chien** : trouve les racines de Λ (positions des erreurs)
4. **Formule de Forney** : calcule les valeurs d'erreur
5. **Correction** : on soustrait les erreurs au mot reçu





r(x)	Received vector
Si	Syndromes
L(x)	Error locator polynomial
Xi	Error locations
Yi	Error magnitudes
c(x)	Recovered code word
v	Number of errors

Algo de Berlekamp–Massey

```
C ← [1], B ← [1], L ← 0, m ← 1
pour r = 0 à 2t − 1 faire
    d ← s_r
    pour i = 1 à L faire
        d ← d ⊕ C[i] · s_{r−i}
    fin pour
    si d ≠ 0 alors
        T ← C
        pour i = 0 à 2t − 1 − m faire
            C[i + m] ← C[i + m] ⊕ d · B[i]
        fin pour
        si 2L ≤ r alors
            L ← r + 1 − L
            B ← d^{−1} · T
            m ← 1
        sinon
            m ← m + 1
        fin si
    sinon
        m ← m + 1
    fin si
fin pour
retourner C tronqué aux L + 1 premiers coefficients
```

$\oplus$ désigne l'addition dans GF(2^m) (XOR).

Recherche de Chien et calcul des valeurs d'erreur

Recherche de Chien

Pour chaque position $i = 0, \ldots, n-1$, on évalue $\Lambda(\alpha^{-i})$ en parallèle via une récurrence.

```
for (int i = 0; i < n; i++) {
int sum = 0;
for (int j = 0; j <= L; j++) sum ^= reg[j];
if (sum == 0) positions.push_back(i);
for (int j = 1; j <= L; j++)
reg[j] = gf.mul(reg[j], alpha_inv[j]);
}
```

Formule de Forney

Pour chaque racine $X_i = \alpha^{-i}$:

$$e_i = -\frac{\Omega(X_i)}{\Lambda'(X_i)}$$

où $\Omega(X) \equiv \Lambda(X)S(X) \pmod{X^{2t}}$ est calculé par multiplication polynomiale.

Complexité et optimisations

- **Précalcul des tables** : $O(n \cdot 2^m)$ en mémoire, mais gain significatif en vitesse.
- **Encodeur** : $O(k \cdot r)$ opérations, chaque itération utilise la table de multiplication précalculée ($O(1)$).
- **Décodeur** :
  - Berlekamp–Massey : $O(t^2)$
  - Chien : $O(n \cdot t)$
  - Forney : $O(t^2)$
- **Choix de q^m** : $q^m = 256$ souvent utilisé (octets) ; ici on utilise 2^m avec m paramétrable.

AFF3CT

AFF3CT

AFF3CT (A Fast Forward Error Correction Tool) est une bibliothèque open source en C++ dédiée aux codes correcteurs d'erreurs. Elle fournit des implémentations optimisées (SIMD, multi-threading) de nombreux codes, dont Reed–Solomon, et sert de référence pour la validation de performances.

Hulotte

Hulotte – outil de validation

L'outil *Hulotte*, développé par notre encadrant de projet, permet de comparer automatiquement notre implémentation avec celle d'AFF3CT. Il exécute les deux encodeurs/décodeurs sur les mêmes sources de données et mesure :

- la correction des résultats (identiques à ceux d'AFF3CT),
- les temps minimal-maximal et moyen d'exécution,

Comparaison

Sélection sudo-luser@DESKTOP-LBENVBS: /mnt/c/Users/Nasr-allah/Desktop/hulotte/rs/build

```
 0,    0,    0,    
```

Résultats expérimentaux

Performances mesurées (moyenne sur 10^4 itérations)

	Notre implémentation	AFF3CT
Encodeur	8.71 μs	10.52 μs
Décodeur	3× plus lent	référence

Analyse

- L'encodeur systématique par LFSR avec précalcul des tables est plus rapide que celui d'AFF3CT (gain de $\approx 17\%$).
- En revanche, le décodeur est 3 fois plus lent. Ce ralentissement provient principalement des étapes de Berlekamp–Massey et de la recherche de Chien, non encore optimisées au niveau matériel.

Pourquoi vectoriser le décodeur ?

- Notre décodeur est 4 fois plus lent que celui d'AFF3CT.
- Les goulots d'étranglement sont :
  - la recherche de Chien (évaluation de $\Lambda(\alpha^{-i})$ sur toutes les positions),
  - l'algorithme de Berlekamp–Massey (multiplications polynomiales).
- Ces boucles traitent des données indépendantes : idéales pour du **calcul parallèle**.

Solution : programmation SIMD

SIMD = *Single Instruction, Multiple Data*.
Une seule instruction opère simultanément sur plusieurs éléments (4, 8, 16...).

MIPP : un wrapper SIMD portable

Problème

Les intrinsèques SIMD (AVX, NEON, SVE) sont différentes selon les processeurs → code non portable.

MIPP (My INTRInsics Plus Plus)

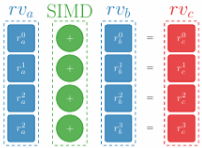
- Bibliothèque C++ développée par AFF3CT.
- Fournit une interface unique : `mipp::Reg<float>` pour un registre vectoriel.
- Le même code compile sur x86 (SSE, AVX, AVX-512), ARM (NEON, SVE), RISC-V.
- Utilisé dans AFF3CT et d'autres codes HPC pour la correction d'erreurs.

Single Instruction Multiple Data (SIMD)

2 Modèles de programmation vectoriel et SIMD

- Instruction scalaire: produit une donnée pendant 1 cycle (pour simplifier)

- Une instruction SIMD produit n données pendant 1 cycle (pour simplifier)



- SIMD = *Single Instruction Multiple Data*
- Les instructions SIMD opèrent sur des registres dit "vectoriels" (rv_a, rv_b, rv_c)

Figure – Source : Cours HPC-CM3-SIMD de Adrien CASSAGNE

Exemple MIPP – Addition matricielle

Principe

Au lieu d'additionner élément par élément, on charge `N` flottants dans des registres SIMD, on additionne en une instruction, puis on stocke. `N = mipp::N<float>()` vaut 4, 8 ou 16 selon le jeu d'instructions (SSE, AVX, AVX-512).

```
constexpr int N = mipp::N<float>();
for (int i = 0; i < rows; ++i)
  for (int j = 0; j < cols - (cols%N); j += N) {
    mipp::Reg<float> rA = mipp::loadu(&A(i,j));
    mipp::Reg<float> rB = mipp::loadu(&B(i,j));
    mipp::Reg<float> rC = rA + rB;
    mipp::storeu(&C(i,j), rC);
  }
// boucle scalaire pour la fin (tail loop)
```

Exemple MIPP – Loi de Poisson (méthode de Knuth)

Algorithme de Knuth (scalaire)

On tire k tel que $\prod_{i=1}^{k} U_i \geq e^{-\lambda}$ et $\prod_{i=1}^{k+1} U_i < e^{-\lambda}$, avec U_i uniformes sur $]0, 1]$.

```
mipp::Reg<float> rL(exp(-lambda));
mipp::Reg<float> p(1.0f);
mipp::Reg<int>   k(0);
mipp::Msk<N> active(true);
while (!mipp::testz(active)) {
  mipp::Reg<float> u = ...;   // N uniformes
  mipp::Msk<N> cond = p > rL;
  p = mipp::blend(p * u, p, cond);
  k = mipp::blend(k + 1, k, cond);
  active = cond;
}
mipp::Reg<int> res = k - 1;
```

Hello World ! : MIPP (commentaires sur les algo's)

- `mipp::loadu` / `storeu` : accès mémoire non aligné (tolère des adresses quelconques).
- Les opérateurs `+`, `-`, `*`, `/` sont surchargés pour `mipp::Reg`.
- On traite `N` éléments à la fois → accélération théorique de `N`.
- Chaque "lane" SIMD exécute indépendamment une instance de l'algorithme.
- `mipp::Msk<N>` : masque booléen (1 = lane active, 0 = terminée).
- `mipp::blend` : choisit entre les deux registres selon le masque.
- `mipp::testz(active)` : vrai si toutes les lanes ont terminé.
- Gain : N variantes de Poisson générées en parallèle.

Merci pour votre attention !

Any Question ? `https://github.com/aff3ct/aff3ct`
`https://github.com/aff3ct/mipp`

Annexe : Dérivation de l'équation-clé

$$S(X) = \sum_{k=1}^{2t} D(\alpha_k) X^{k-1} = \sum_{k=1}^{2t} \sum_{h=1}^{\nu} e_{i_h} \alpha_k^{i_h} X^{k-1}$$
$$= \sum_{h=1}^{\nu} e_{i_h} \alpha^{i_h} \sum_{k=1}^{2t} (\alpha^{i_h} X)^{k-1} = \sum_{h=1}^{\nu} e_{i_h} \alpha^{i_h} \frac{1 - (\alpha^{i_h} X)^{2t}}{1 - \alpha^{i_h} X}.$$

Alors

$$\Lambda(X) S(X) = \sum_{h=1}^{\nu} e_{i_h} \alpha^{i_h} \left(1 - (\alpha^{i_h} X)^{2t}\right) \prod_{\substack{k=1 \\ k \neq h}}^{\nu} (1 - \alpha^{i_k} X)$$
$$\equiv \sum_{h=1}^{\nu} e_{i_h} \alpha^{i_h} \prod_{k \neq h} (1 - \alpha^{i_k} X) \pmod{X^{2t}}$$
$$= \Omega(X).$$

Formule de Forney

$$\Omega(\alpha^{-i_k}) = e_{i_k}\alpha^{i_k}\prod_{j\neq k}(1-\alpha^{i_j}\alpha^{-i_k})$$

$$\Lambda'(\alpha^{-i_k}) = \sum_{h=1}^{\nu}(-\alpha^{i_h})\prod_{j\neq h}(1-\alpha^{i_j}\alpha^{-i_k}) = -\alpha^{i_k}\prod_{j\neq k}(1-\alpha^{i_j}\alpha^{-i_k})$$

d'où

$$e_{i_k} = -\frac{\Omega(\alpha^{-i_k})}{\Lambda'(\alpha^{-i_k})}.$$